

TOPIC 1

1. Given an array of strings `words`, return the first palindromic string in the array. If there is no such string, return an empty string `""`. A string is palindromic if it reads the same forward and backward.

Example 1:

Input: `words = ["abc","car","ada","racecar","cool"]`

Output: `"ada"`

Explanation: The first string that is palindromic is `"ada"`.

Note that `"racecar"` is also palindromic, but it is not the first.

Example 2:

Input: `words = ["notapalindrome","racecar"]`

Output: `"racecar"`

Explanation: The first and only string that is palindromic is `"racecar"`.

Aim

To write a C program that finds and returns the first palindromic string from an array of strings. If no palindrome exists, print an empty string.

Algorithm

1. Start.
2. Read the number of strings `n`.
3. Input all the strings into an array.
4. For each string:
 - Compare characters from the beginning and the end.
 - If all corresponding characters match, the string is a palindrome.
 - Print the string and stop the program.
5. If no palindrome is found after checking all strings, print an empty string `""`.
6. Stop.

CODE

```
1 words = input("Enter words separated by space: ").split()
2
3 for word in words:
4     if word == word[::-1]:
5         print("First Palindromic String:", word)
6         break
7 else:
8     print("")
```

OUTPUT

Output

```
Enter words separated by space: abc car ada racecar cool
First Palindromic String: ada
```

```
=== Code Execution Successful ===
```

Time Complexity

- Let **N** be the number of strings and **M** be the maximum length of a string.
- Checking one string takes **O(M)**.
- Checking all strings takes **O(N × M)**.

Time Complexity: $O(N \times M)$

Space Complexity

- Only a few extra variables are used for checking the palindrome.

Auxiliary Space Complexity: $O(1)$

Overall Space (including input storage): $O(N \times M)$

2. You are given two integer arrays nums1 and nums2 of sizes n and m, respectively. Calculate the following values: answer1 : the number of indices i such that nums1[i] exists in nums2. answer2 : the number of indices i such that nums2[i] exists in nums1 Return [answer1,answer2].

Example 1:

Input: nums1 = [2,3,2], nums2 = [1,2]

Output: [2,1]

Explanation:

Example 2:

Input: nums1 = [4,3,2,3,1], nums2 = [2,2,5,2,3,6]

Output: [3,4]

Explanation:

The elements at indices 1, 2, and 3 in nums1 exist in nums2 as well. So answer1 is 3.

The elements at indices 0, 1, 3, and 4 in nums2 exist in nums1. So answer2 is 4.

Aim

To write a Python program to count how many elements of one array exist in the other array and return [answer1, answer2].

Algorithm

1. Read two integer arrays.
2. Convert both arrays into sets.
3. Count elements in nums1 that exist in nums2.
4. Count elements in nums2 that exist in nums1.
5. Print the counts as a list.

CODE

```
main.py  [ ] [ ] [ ] Share Run
1  nums1 = list(map(int, input("Enter elements of nums1: ").split()))
2  nums2 = list(map(int, input("Enter elements of nums2: ").split()))
3
4  answer1 = 0
5  answer2 = 0
6
7  for i in nums1:
8      if i in nums2:
9          answer1 += 1
10
11 for i in nums2:
12     if i in nums1:
13         answer2 += 1
14
15 print([answer1, answer2])
```

OUTPUT

```
Output
Enter elements of nums1: 2 3 2
Enter elements of nums2: 1 2
[2, 1]

=== Code Execution Successful ===
```

Time Complexity

$O(n + m)$

Space Complexity

$O(n + m)$

3. You are given a 0-indexed integer array `nums`. The distinct count of a subarray of `nums` is defined as: Let `nums[i..j]` be a subarray of `nums` consisting of all the indices from `i` to `j` such that $0 \leq i \leq j < \text{nums.length}$. Then the number of distinct values in `nums[i..j]` is called the distinct count of `nums[i..j]`. Return the sum of the squares of distinct counts of all subarrays of `nums`. A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

Input: `nums = [1,2,1]`

Output: 15

Explanation: Six possible subarrays are:

`[1]`: 1 distinct value

`[2]`: 1 distinct value

`[1]`: 1 distinct value

`[1,2]`: 2 distinct values

`[2,1]`: 2 distinct values

`[1,2,1]`: 2 distinct values

The sum of the squares of the distinct counts in all subarrays is equal to $1^2 + 1^2 + 1^2 + 2^2 + 2^2 + 2^2 = 15$.

Example 2:

Input: `nums = [1,1]`

Output: 3

Explanation: Three possible subarrays are:

`[1]`: 1 distinct value

`[1]`: 1 distinct value

`[1,1]`: 1 distinct value

The sum of the squares of the distinct counts in all subarrays is equal to $1^2 + 1^2 + 1^2 = 3$.

Aim

To write a Python program to find the sum of the squares of the number of distinct elements in all subarrays.

Algorithm

1. Read the array.
2. Generate all possible subarrays.
3. For each subarray, find the number of distinct elements using a set.

4. Square the distinct count and add it to the total.
5. Print the total sum.

CODE

```
1  nums = list(map(int, input("Enter array elements: ")
2      ).split()))
3  total = 0
4  n = len(nums)
5
6  for i in range(n):
7      s = set()
8      for j in range(i, n):
9          s.add(nums[j])
10         distinct = len(s)
11         total += distinct * distinct
12
13 print(total)
```

OUTPUT

Output
Enter array elements: 1 2 1
15
=== Code Execution Successful ===

Time Complexity

$O(n^2)$

Space Complexity

$O(n)$

4. Given a 0-indexed integer array `nums` of length `n` and an integer `k`, return the number of pairs (i, j) where $0 \leq i < j < n$, such that `nums[i] == nums[j]` and $(i * j)$ is divisible by `k`.

Example 1:

Input: `nums = [3,1,2,2,2,1,3]`, `k = 2`

Output: 4

Explanation:

There are 4 pairs that meet all the requirements:

- `nums[0] == nums[6]`, and $0 * 6 == 0$, which is divisible by 2.
- `nums[2] == nums[3]`, and $2 * 3 == 6$, which is divisible by 2.
- `nums[2] == nums[4]`, and $2 * 4 == 8$, which is divisible by 2.
- `nums[3] == nums[4]`, and $3 * 4 == 12$, which is divisible by 2.

Example 2:

Input: `nums = [1,2,3,4]`, `k = 1`

Output: 0

Explanation: Since no value in `nums` is repeated, there are no pairs (i, j) that meet all the requirements.

Aim

To write a Python program to count the number of pairs (i, j) such that `nums[i] == nums[j]` and $(i * j)$ is divisible by `k`.

Algorithm

1. Start the program.
2. Read the array elements from the user.
3. Read the value of `k`.
4. Initialize a variable `count = 0`.
5. Find the length of the array.
6. Use two nested loops:
 - Outer loop from `i = 0` to `n-1`.

- Inner loop from $j = i+1$ to $n-1$.
- 7. Check if:
 - `nums[i] == nums[j]`, and
 - `(i * j) % k == 0`.
- 8. If both conditions are true, increment `count` by 1.
- 9. Display the value of `count`.
- 10. Stop the program.

CODE

```
1  nums = list(map(int, input("Enter array elements: ").split
    ()))
2  k = int(input("Enter k: "))
3
4  count = 0
5  n = len(nums)
6
7  for i in range(n):
8      for j in range(i + 1, n):
9          if nums[i] == nums[j] and (i * j) % k == 0:
10             count += 1
11
12  print("Output:", count)
13
```

OUTPUT

```
Output
Enter array elements: 3 1 2 2 2 1 3
Enter k: 1
Output: 5

=== Code Execution Successful ===
```

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

5. Write a program FOR THE BELOW TEST CASES with least time complexity

Test Cases: -

Input: {1, 2, 3, 4, 5} Expected Output: 5

Input: {7, 7, 7, 7, 7} Expected Output: 7

Input: {-10, 2, 3, -4, 5} Expected Output: 5

Aim

To write a Python program to find the **largest element** in an array with the least time complexity.

Algorithm

1. Start the program.
2. Read the array elements from the user.
3. Assume the first element is the largest.
4. Traverse the array from the second element to the last.
5. If the current element is greater than the largest element, update the largest element.
6. Display the largest element.
7. Stop the program.

CODE

```
1  nums = list(map(int, input("Enter array elements: ").split()))
2
3  largest = nums[0]
4
5  for i in nums:
6      if i > largest:
7          largest = i
8
9  print("Output:", largest)
```

OUTPUT

Output

Enter array elements: 1 2 3 4 5

Output: 5

=== Code Execution Successful ===

Time Complexity: $O(n)$

Space Complexity: $O(1)$

6. You have an algorithm that process a list of numbers. It firsts sorts the list using an efficient sorting algorithm and then finds the maximum element in sorted list. Write the code for the same.

Test Cases

1. Empty List

1. Input: []

2. Expected Output: None or an appropriate message indicating that the list is empty.

2. Single Element List

1. Input: [5]

2. Expected Output: 5

3. All Elements are the Same

1. Input: [3, 3, 3, 3, 3]

2. Expected Output: 3

Aim

To write a Python program to sort a list using an efficient sorting algorithm and then find the maximum element in the sorted list.

Algorithm

1. Start the program.
2. Read the list of numbers from the user.
3. Check if the list is empty.
4. If the list is empty, display **"List is empty"** and stop.
5. Otherwise, sort the list in ascending order using the built-in `sort()` method.
6. The last element of the sorted list is the maximum element.
7. Display the maximum element.

8. Stop the program.

CODE

```
1 nums = list(map(int, input("Enter array elements: ").split()))
2
3 if len(nums) == 0:
4     print("List is empty")
5 else:
6     nums.sort()
7     print("Sorted List:", nums)
8     print("Maximum Element:", nums[-1])
```

OUTPUT

```
Output
Enter array elements: 5
Sorted List: [5]
Maximum Element: 5

=== Code Execution Successful ===
```

Time Complexity: $O(n \log n)$ (Sorting)

Space Complexity: $O(1)$ (ignoring Python's internal sort space)

7. Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm?

Test Cases

Some Duplicate Elements

Input: [3, 7, 3, 5, 2, 5, 9, 2]

Expected Output: [3, 7, 5, 2, 9] (Order may vary based on the algorithm used)

Negative and Positive Numbers

Input: [-1, 2, -1, 3, 2, -2]

Expected Output: [-1, 2, 3, -2] (Order may vary)

List with Large Numbers

Input: [1000000, 999999, 1000000]

Expected Output: [1000000, 999999]

Aim

To write a Python program to create a new list containing only the unique elements from the given list and determine its space complexity.

Algorithm

1. Start the program.
2. Read the list of numbers from the user.
3. Create an empty list called **unique**.
4. Traverse each element in the input list.
5. If the element is not present in **unique**, add it to **unique**.
6. Display the **unique** list.
7. Stop the program.

Code

```
1 nums = list(map(int, input("Enter array elements: ").split()))
2 unique = []
3 for i in nums:
4     if i not in unique:
5         unique.append(i)
6 print("Unique Elements:", unique)
7
8
```

Output

```
Enter array elements: 3 5 2 3 5 9 2
Unique Elements: [3, 5, 2, 9]

=== Code Execution Successful ===
```

Time Complexity

- $O(n^2)$ (because `i not in unique` searches the list linearly)

Space Complexity

- $O(n)$ (in the worst case, all elements are unique, so the new list stores all n elements)

8. Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code

Aim

To write a Python program to sort an array of integers using the **Bubble Sort** technique and analyze its time complexity.

Algorithm

1. Start the program.
2. Read the array elements from the user.
3. Find the length of the array.
4. Use two nested loops:
 - Outer loop from 0 to $n-1$.
 - Inner loop from 0 to $n-i-2$.
5. Compare adjacent elements.
6. If the left element is greater than the right element, swap them.
7. Repeat until the array is sorted.
8. Display the sorted array.
9. Stop the program.

Code

```
1 nums = list(map(int, input("Enter array elements: ").split()))
2 n = len(nums)
3 for i in range(n):
4     for j in range(n - i - 1):
5         if nums[j] > nums[j + 1]:
6             nums[j], nums[j + 1] = nums[j + 1], nums[j]
7 print("Sorted Array:", nums)
8
```

Output

```
Output
Enter array elements: 5 1 4 2 8
Sorted Array: [1, 2, 4, 5, 8]

=== Code Execution Successful ===
```

Time Complexity (Big-O)

- Best Case: $O(n)$ (with optimization for already sorted arrays)
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

Space Complexity

- $O(1)$ (In-place sorting)

9. Checks if a given number x exists in a sorted array arr using binary search.

Analyze its time complexity using Big-O notation.

Test Case:

Example X={ 3,4,6,-9,10,8,9,30} KEY=10

Output: Element 10 is found at position 5

Example X={ 3,4,6,-9,10,8,9,30} KEY=100

Output : Element 100 is not found

Aim

To write a Python program to search for a given element in a sorted array using **Binary Search** and analyze its time complexity.

Algorithm

1. Start the program.
2. Read the array elements from the user.
3. Sort the array.
4. Read the key element to be searched.
5. Initialize low = 0 and high = n - 1.
6. Repeat while low <= high:
 - Find the middle index mid = (low + high) // 2.
 - If arr[mid] == key, display its position and stop.
 - If key < arr[mid], set high = mid - 1.
 - Otherwise, set low = mid + 1.
7. If the element is not found, display "**Element not found**".
8. Stop the program.

Code

```
1 arr = list(map(int, input("Enter array elements: ").split()))
2 key = int(input("Enter key: "))
3 arr.sort()
4 low = 0
5 high = len(arr) - 1
6 found = False
7 while low <= high:
8     mid = (low + high) // 2
9
10    if arr[mid] == key:
11        print("Sorted Array:", arr)
12        print("Element", key, "is found at position", mid + 1)
13        found = True
14        break
15    elif key < arr[mid]:
16        high = mid - 1
17    else:
18        low = mid + 1
19 if not found:
20     print("Sorted Array:", arr)
21     print("Element", key, "is not found")
22
```

Output

Output

```
Enter array elements: 3 4 6 -9 10 8 9 30
Enter key: 10
Sorted Array: [-9, 3, 4, 6, 8, 9, 10, 30]
Element 10 is found at position 7
```

```
=== Code Execution Successful ===
```

Time Complexity (Big-O)

- Best Case: $O(1)$
- Average Case: $O(\log n)$
- Worst Case: $O(\log n)$

Space Complexity

- $O(1)$

10. Given an array of integers nums, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in $O(n \log(n))$ time complexity and with the smallest space complexity possible.

Aim

To sort an array of integers in **ascending order** without using any built-in sorting functions by implementing the **Merge Sort** algorithm, which runs in **$O(n \log n)$** time complexity.

Algorithm

1. Read the size of the array n.
2. Read the array elements.
3. Divide the array into two halves recursively until each subarray contains one element.
4. Merge the subarrays by comparing elements and arranging them in ascending order.
5. Repeat the merge process until the complete array is sorted.
6. Display the sorted array.

Code

```
1 def merge_sort(arr):
2     if len(arr) > 1:
3         mid = len(arr) // 2
4         left = arr[:mid]
5         right = arr[mid:]
6         merge_sort(left)
7         merge_sort(right)
8         i = j = k = 0
9         while i < len(left) and j < len(right):
10            if left[i] < right[j]:
11                arr[k] = left[i]
12                i += 1
13            else:
14                arr[k] = right[j]
15                j += 1
16            k += 1
17        while i < len(left):
18            arr[k] = left[i]
19            i += 1
20            k += 1
21        while j < len(right):
22            arr[k] = right[j]
23            j += 1
24            k += 1
```

Output

Output

Enter the number of elements: 5

Enter the elements:

5

2

3

1

4

Sorted array: [1, 2, 3, 4, 5]

=== Code Execution Successful ===

Time Complexity

- **Best Case:** $O(n \log n)$
- **Average Case:** $O(n \log n)$
- **Worst Case:** $O(n \log n)$

Space Complexity

- **$O(n)$** (temporary arrays used during merging)

11. Given an $m \times n$ grid and a ball at a starting cell, find the number of ways to move the ball out of the grid boundary in exactly N steps.

Example:

- **Input:** $m=2, n=2, N=2, i=0, j=0$ · **Output:** 6
- **Input:** $m=1, n=3, N=3, i=0, j=1$ · **Output:** 12

Aim

To find the number of ways to move a ball out of an $m \times n$ grid boundary in exactly N steps using **Dynamic Programming (DP)**.

Algorithm

1. Read the values of **m , n , N , starting row (i), and starting column (j)**.
2. Use a recursive function with memoization (DP).
3. If the ball moves outside the grid, return **1**.
4. If no steps are left and the ball is still inside the grid, return **0**.
5. From the current cell, move in four directions:
 - Up
 - Down
 - Left
 - Right
6. Sum all possible ways.
7. Store intermediate results in the DP table to avoid recomputation.
8. Print the total number of ways.\

Code

```
1 def paths(m, n, N, i, j):
2     if i < 0 or i >= m or j < 0 or j >= n:
3         return 1
4     if N == 0:
5         return 0
6
7     return (paths(m, n, N-1, i-1, j) +
8           paths(m, n, N-1, i+1, j) +
9           paths(m, n, N-1, i, j-1) +
10          paths(m, n, N-1, i, j+1))
11
12 # User Input
13 m = int(input("Enter rows: "))
14 n = int(input("Enter columns: "))
15 N = int(input("Enter steps: "))
16 i = int(input("Enter start row: "))
17 j = int(input("Enter start column: "))
18
19 print("Number of ways:", paths(m, n, N, i, j))
20
```

Output

```
Enter columns: 3
Enter steps: 3
Enter start row: 0
Enter start column: 1
Output
Number of ways: 12|
```

Time Complexity

- $O(m \times n \times N)$

Space Complexity

- $O(m \times n \times N)$ (for DP table)

12. You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are arranged in a circle. That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have security systems connected, and it will automatically contact the police if two adjacent houses were broken into on the same night.

Examples:

Input : nums = [2, 3, 2]

Output : The maximum money you can rob without alerting the police is 3 (robbing house 1).

(ii) Input : nums = [1, 2, 3, 1]

Output : The maximum money you can rob without alerting the police is 4 (robbing house 1 and house 3).

Aim

To find the **maximum amount of money** that can be robbed from houses arranged in a **circle** without robbing two adjacent houses using **Dynamic Programming**.

Algorithm

1. Read the number of houses n.
2. Read the money in each house.
3. If there is only one house, return its value.
4. Since the houses are arranged in a circle:
 - Find the maximum money by robbing houses from **0 to n-2**.
 - Find the maximum money by robbing houses from **1 to n-1**.
5. Use Dynamic Programming for each case:
 - $dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$
6. Return the maximum of the two cases.
7. Display the result.

Code

```
1 def rob(nums):
2     if len(nums) == 1:
3         return nums[0]
4
5     def solve(arr):
6         prev = curr = 0
7         for x in arr:
8             prev, curr = curr, max(curr, prev + x)
9         return curr
10
11     return max(solve(nums[:-1]), solve(nums[1:]))
12
13 # User Input
14 n = int(input("Enter number of houses: "))
15 nums = []
16
17 print("Enter money in each house:")
18 for i in range(n):
19     nums.append(int(input()))
20
21 print("Maximum money that can be robbed:", rob(nums))
```

Output

```
Enter number of houses: 3
Enter money in each house:
2
3
2
Maximum money that can be robbed: 3

=== Code Execution Successful ===
```

Time Complexity

- $O(n)$

Space Complexity

- $O(1)$

13. You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Examples:

Input: $n=4$ Output: 5

Input: $n=3$ Output: 3

Aim

To find the **number of distinct ways** to climb a staircase with **n** steps, where each time you can climb either **1 step or 2 steps**, using **Dynamic Programming**.

Algorithm

1. Read the number of steps n .
2. If $n = 1$, return 1.
3. If $n = 2$, return 2.
4. Initialize:
 - $first = 1$
 - $second = 2$
5. For each step from 3 to n :
 - $current = first + second$
 - Update $first = second$
 - Update $second = current$
6. Print $second$ as the total number of distinct ways.

Code

```
1 def climb(n):
2     if n <= 2:
3         return n
4
5     a, b = 1, 2
6     for i in range(3, n + 1):
7         a, b = b, a + b
8     return b
9
10 # User Input
11 n = int(input("Enter number of steps: "))
12
13 print("Number of ways:", climb(n))
14
```

Time Complexity

- $O(n)$

Space Complexity

- $O(1)$

14. A robot is located at the top-left corner of a $m \times n$ grid. The robot can only move either down or right at any point in time. The robot is trying to reach the bottom-right corner of the grid. How many possible unique paths are there?

Examples:

Input: $m=7, n=3$ **Output:** 28

Input: $m=3, n=2$ **Output:** 3

Aim

To find the **number of unique paths** for a robot to travel from the **top-left corner** to the **bottom-right corner** of an $m \times n$ grid, where the robot can move only **right** or **down**, using **Dynamic Programming**.

Algorithm

1. Read the values of **m** and **n**.
2. Create a DP table of size $m \times n$.
3. Initialize the first row and first column with 1 since there is only one way to reach those cells.
4. For each remaining cell, compute:
 - o $dp[i][j] = dp[i-1][j] + dp[i][j-1]$
5. The value at $dp[m-1][n-1]$ is the total number of unique paths.

6. Print the result.

CODE

```
1 import math
2
3 m = int(input("Enter m: "))
4 n = int(input("Enter n: "))
5
6 print("Unique Paths =", math.comb(m+n-2, m-1))
```

OUTPUT

Output
Enter m: 7 Enter n: 3 Unique Paths = 28 === Code Execution Successful ===

Time Complexity

- $O(m \times n)$

Space Complexity

- $O(m \times n)$

15.In a string *S* of lowercase letters, these letters form consecutive groups of the same character. For example, a string like *s* = "abbxxxxzzy" has the groups "a", "bb", "xxxx", "z", and "yy". A group is identified by an interval [start, end], where start and end denote the start and end indices (inclusive) of the group. In the above example, "xxxx" has the interval [3,6]. A group is considered large if it has 3 or more characters. Return the intervals of every large group sorted in increasing order by start index.

Example 1:

Input: *s* = "abbxxxxzzy"

Output: [[3,6]]

Explanation: "xxxx" is the only large group with start index 3 and end index 6.

Example 2:

Input: *s* = "abc"

Output: []

Explanation: We have groups "a", "b", and "c", none of which are large groups.

Aim

To find all **large groups** (consecutive groups of the same character having length **3 or more**) in a given string and print their **start and end indices**.

Algorithm

1. Read the input string S.
2. Initialize start = 0.
3. Traverse the string from index 0 to the last character.
4. Whenever the current character is different from the next character (or the end of the string is reached):
 - o Calculate the group length.
 - o If the length is **3 or more**, print the interval [start, end].
 - o Update start to the next index.
5. If no large groups are found, print [].

Code

```
1 s = input("Enter string: ")
2 r = []
3 i = 0
4 while i < len(s):
5     j = i
6     while j < len(s) and s[j] == s[i]:
7         j += 1
8     if j - i >= 3:
9         r.append([i, j - 1])
10    i = j
11 print(r)
```

Output

```
Enter string: abbxxxxzy
```

```
[[3, 6]]
```

```
=== Code Execution Successful ===
```

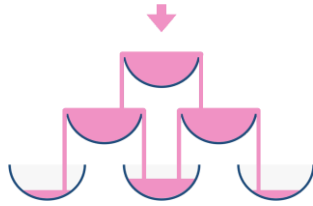
Time Complexity

$O(n)$

Space Complexity

$O(1)$

16. We stack glasses in a pyramid, where the first row has 1 glass, the second row has 2 glasses, and so on until the 100th row. Each glass holds one cup of champagne. Then, some champagne is poured into the first glass at the top. When the topmost glass is full, any excess liquid poured will fall equally to the glass immediately to the left and right of it. When those glasses become full, any excess champagne will fall equally to the left and right of those glasses, and so on. (A glass at the bottom row has its excess champagne fall on the floor.) For example, after one cup of champagne is poured, the top most glass is full. After two cups of champagne are poured, the two glasses on the second row are half full. After three cups of champagne are poured, those two cups become full - there are 3 full glasses total now. After four cups of champagne are poured, the third row has the middle glass half full, and the two outside glasses are a quarter full, as pictured below.



Now after pouring some non-negative integer cups of champagne, return how full the j^{th} glass in the i^{th} row is (both i and j are 0-indexed.)

Example 1:

Input: poured = 1, query_row = 1, query_glass = 1

Output: 0.00000

Explanation: We poured 1 cup of champagne to the top glass of the tower (which is indexed as (0, 0)). There will be no excess liquid so all the glasses under the top glass will remain empty. **Example 2:**

Input: poured = 2, query_row = 1, query_glass = 1

Output: 0.50000

Explanation: We poured 2 cups of champagne to the top glass of the tower (which is indexed as (0, 0)). There is one cup of excess liquid. The glass indexed as (1, 0) and the glass indexed as (1, 1) will share the excess liquid equally, and each will get half cup of champagne.

Aim

To determine how full a specific glass is after pouring a given number of cups of champagne into the top glass of a champagne tower using **Dynamic Programming**.

Algorithm

1. Read the values of **poured**, **query_row**, and **query_glass**.
2. Create a 2D array `dp[101][101]` initialized to 0.
3. Pour all the champagne into the top glass: `dp[0][0] = poured`.
4. For each row up to `query_row`:
 - o If a glass contains more than **1 cup**, calculate the excess:
 - $\text{excess} = (\text{dp}[i][j] - 1) / 2$
 - o Add the excess equally to the left and right glasses in the next row.
5. The required glass contains `min(1.0, dp[query_row][query_glass])`.
6. Print the result.

Code

```

1 p = int(input("Poured: "))
2 r = int(input("Query Row: "))
3 g = int(input("Query Glass: "))
4 a = [[0]*101 for _ in range(101)]
5 a[0][0] = p
6 for i in range(r+1):
7     for j in range(i+1):
8         x = max(0, (a[i][j]-1)/2)
9         a[i+1][j] += x
10        a[i+1][j+1] += x
11 print(f"{min(1, a[r][g]):.5f}")
12

```

Output

```

Poured: 1
Query Row: 1
Query Glass: 1
0.00000

=== Code Execution Successful ===

```

Time Complexity

$O(\text{query_row}^2)$

Space Complexity

$O(101 \times 101) \approx O(1)$ (fixed-size DP table)